

P2093R11

Formatted output

Published Proposal, 2021-12-06

Author:

[Victor Zverovich](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Table of Contents

1	Introduction	
2	Revision history	
3	SG16 polls (R8)	
4	SG16 polls (R6)	
5	LEWG polls (R5)	
6	SG16 polls (R3)	
7	LEWG polls (R2)	
8	LEWG polls (R1)	
9	Motivating examples	
10	API and naming	
11	Unicode	
12	Performance	
13	Binary code	
14	Impact on existing code	
15	Implementation	
16	Wording	
	Appendix A: Unicode tests	
17	Acknowledgements	
	References	
	Informative References	

"𐀀𐀁𐀂𐀃𐀄𐀅𐀆𐀇𐀈𐀉𐀊𐀋𐀌𐀍𐀎𐀏𐀐𐀑𐀒𐀓𐀔𐀕𐀖𐀗𐀘𐀙𐀚𐀛𐀜𐀝𐀞𐀟𐀠𐀡𐀢𐀣𐀤𐀥𐀦𐀧𐀨𐀩𐀪𐀫𐀬𐀭𐀮𐀯𐀰𐀱𐀲𐀳𐀴𐀵𐀶𐀷𐀸𐀹𐀺𐀻𐀼𐀽𐀾𐀿𐁀𐁁𐁂𐁃𐁄𐁅𐁆𐁇𐁈𐁉𐁊𐁋𐁌𐁍𐁎𐁏𐁐𐁑𐁒𐁓𐁔𐁕𐁖𐁗𐁘𐁙𐁚𐁛𐁜𐁝𐁞𐁟𐁠𐁡𐁢𐁣𐁤𐁥𐁦𐁧𐁨𐁩𐁪𐁫𐁬𐁭𐁮𐁯𐁰𐁱𐁲𐁳𐁴𐁵𐁶𐁷𐁸𐁹𐁺𐁻𐁼𐁽𐁾𐁿𐂀𐂁𐂂𐂃𐂄𐂅𐂆𐂇𐂈𐂉𐂊𐂋𐂌𐂍𐂎𐂏𐂐𐂑𐂒𐂓𐂔𐂕𐂖𐂗𐂘𐂙𐂚𐂛𐂜𐂝𐂞𐂟𐂠𐂡𐂢𐂣𐂤𐂥𐂦𐂧𐂨𐂩𐂪𐂫𐂬𐂭𐂮𐂯𐂰𐂱𐂲𐂳𐂴𐂵𐂶𐂷𐂸𐂹𐂺𐂻𐂼𐂽𐂾𐂿𐃀𐃁𐃂𐃃𐃄𐃅𐃆𐃇𐃈𐃉𐃊𐃋𐃌𐃍𐃎𐃏𐃐𐃑𐃒𐃓𐃔𐃕𐃖𐃗𐃘𐃙𐃚𐃛𐃜𐃝𐃞𐃟𐃠𐃡𐃢𐃣𐃤𐃥𐃦𐃧𐃨𐃩𐃪𐃫𐃬𐃭𐃮𐃯𐃰𐃱𐃲𐃳𐃴𐃵𐃶𐃷𐃸𐃹𐃺𐃻𐃼𐃽𐃾𐃿𐄀𐄁𐄂𐄃𐄄𐄅𐄆𐄇𐄈𐄉𐄊𐄋𐄌𐄍𐄎𐄏𐄐𐄑𐄒𐄓𐄔𐄕𐄖𐄗𐄘𐄙𐄚𐄛𐄜𐄝𐄞𐄟𐄠𐄡𐄢𐄣𐄤𐄥𐄦𐄧𐄨𐄩𐄪𐄫𐄬𐄭𐄮𐄯𐄰𐄱𐄲𐄳𐄴𐄵𐄶𐄷𐄸𐄹𐄺𐄻𐄼𐄽𐄾𐄿𐅀𐅁𐅂𐅃𐅄𐅅𐅆𐅇𐅈𐅉𐅊𐅋𐅌𐅍𐅎𐅏𐅐𐅑𐅒𐅓𐅔𐅕𐅖𐅗𐅘𐅙𐅚𐅛𐅜𐅝𐅞𐅟𐅠𐅡𐅢𐅣𐅤𐅥𐅦𐅧𐅨𐅩𐅪𐅫𐅬𐅭𐅮𐅯𐅰𐅱𐅲𐅳𐅴𐅵𐅶𐅷𐅸𐅹𐅺𐅻𐅼𐅽𐅾𐅿𐆀𐆁𐆂𐆃𐆄𐆅𐆆𐆇𐆈𐆉𐆊𐆋𐆌𐆍𐆎𐆏𐆐𐆑𐆒𐆓𐆔𐆕𐆖𐆗𐆘𐆙𐆚𐆛𐆜𐆝𐆞𐆟𐆠𐆡𐆢𐆣𐆤𐆥𐆦𐆧𐆨𐆩𐆪𐆫𐆬𐆭𐆮𐆯𐆰𐆱𐆲𐆳𐆴𐆵𐆶𐆷𐆸𐆹𐆺𐆻𐆼𐆽𐆾𐆿𐇀𐇁𐇂𐇃𐇄𐇅𐇆𐇇𐇈𐇉𐇊𐇋𐇌𐇍𐇎𐇏𐇐𐇑𐇒𐇓𐇔𐇕𐇖𐇗𐇘𐇙𐇚𐇛𐇜𐇝𐇞𐇟𐇠𐇡𐇢𐇣𐇤𐇥𐇦𐇧𐇨𐇩𐇪𐇫𐇬𐇭𐇮𐇯𐇰𐇱𐇲𐇳𐇴𐇵𐇶𐇷𐇸𐇹𐇺𐇻𐇼𐇽𐇾𐇿𐈀𐈁𐈂𐈃𐈄𐈅𐈆𐈇𐈈𐈉𐈊𐈋𐈌𐈍𐈎𐈏𐈐𐈑𐈒𐈓𐈔𐈕𐈖𐈗𐈘𐈙𐈚𐈛𐈜𐈝𐈞𐈟𐈠𐈡𐈢𐈣𐈤𐈥𐈦𐈧𐈨𐈩𐈪𐈫𐈬𐈭𐈮𐈯𐈰𐈱𐈲𐈳𐈴𐈵𐈶𐈷𐈸𐈹𐈺𐈻𐈼𐈽𐈾𐈿𐉀𐉁𐉂𐉃𐉄𐉅𐉆𐉇𐉈𐉉𐉊𐉋𐉌𐉍𐉎𐉏𐉐𐉑𐉒𐉓𐉔𐉕𐉖𐉗𐉘𐉙𐉚𐉛𐉜𐉝𐉞𐉟𐉠𐉡𐉢𐉣𐉤𐉥𐉦𐉧𐉨𐉩𐉪𐉫𐉬𐉭𐉮𐉯𐉰𐉱𐉲𐉳𐉴𐉵𐉶𐉷𐉸𐉹𐉺𐉻𐉼𐉽𐉾𐉿𐊀𐊁𐊂𐊃𐊄𐊅𐊆𐊇𐊈𐊉𐊊𐊋𐊌𐊍𐊎𐊏𐊐𐊑𐊒𐊓𐊔𐊕𐊖𐊗𐊘𐊙𐊚𐊛𐊜𐊝𐊞𐊟𐊠𐊡𐊢𐊣𐊤𐊥𐊦𐊧𐊨𐊩𐊪𐊫𐊬𐊭𐊮𐊯𐊰𐊱𐊲𐊳𐊴𐊵𐊶𐊷𐊸𐊹𐊺𐊻𐊼𐊽𐊾𐊿𐋀𐋁𐋂𐋃𐋄𐋅𐋆𐋇𐋈𐋉𐋊𐋋𐋌𐋍𐋎𐋏𐋐𐋑𐋒𐋓𐋔𐋕𐋖𐋗𐋘𐋙𐋚𐋛𐋜𐋝𐋞𐋟𐋠𐋡𐋢𐋣𐋤𐋥𐋦𐋧𐋨𐋩𐋪𐋫𐋬𐋭𐋮𐋯𐋰𐋱𐋲𐋳𐋴𐋵𐋶𐋷𐋸𐋹𐋺𐋻𐋼𐋽𐋾𐋿𐌀𐌁𐌂𐌃𐌄𐌅𐌆𐌇𐌈𐌉𐌊𐌋𐌌𐌍𐌎𐌏𐌐𐌑𐌒𐌓𐌔𐌕𐌖𐌗𐌘𐌙𐌚𐌛𐌜𐌝𐌞𐌟𐌠𐌡𐌢𐌣𐌤𐌥𐌦𐌧𐌨𐌩𐌪𐌫𐌬𐌭𐌮𐌯𐌰𐌱𐌲𐌳𐌴𐌵𐌶𐌷𐌸𐌹𐌺𐌻𐌼𐌽𐌾𐌿𐍀𐍁𐍂𐍃𐍄𐍅𐍆𐍇𐍈𐍉𐍊𐍋𐍌𐍍𐍎𐍏𐍐𐍑𐍒𐍓𐍔𐍕𐍖𐍗𐍘𐍙𐍚𐍛𐍜𐍝𐍞𐍟𐍠𐍡𐍢𐍣𐍤𐍥𐍦𐍧𐍨𐍩𐍪𐍫𐍬𐍭𐍮𐍯𐍰𐍱𐍲𐍳𐍴𐍵𐍶𐍷𐍸𐍹𐍺𐍻𐍼𐍽𐍾𐍿𐎀𐎁𐎂𐎃𐎄𐎅𐎆𐎇𐎈𐎉𐎊𐎋𐎌𐎍𐎎𐎏𐎐𐎑𐎒𐎓𐎔𐎕𐎖𐎗𐎘𐎙𐎚𐎛𐎜𐎝𐎞𐎟𐎠𐎡𐎢𐎣𐎤𐎥𐎦𐎧𐎨𐎩𐎪𐎫𐎬𐎭𐎮𐎯𐎰𐎱𐎲𐎳𐎴𐎵𐎶𐎷𐎸𐎹𐎺𐎻𐎼𐎽𐎾𐎿𐏀𐏁𐏂𐏃𐏄𐏅𐏆𐏇𐏈𐏉𐏊𐏋𐏌𐏍𐏎𐏏𐏐𐏑𐏒𐏓𐏔𐏕𐏖𐏗𐏘𐏙𐏚𐏛𐏜𐏝𐏞𐏟𐏠𐏡𐏢𐏣𐏤𐏥𐏦𐏧𐏨𐏩𐏪𐏫𐏬𐏭𐏮𐏯𐏰𐏱𐏲𐏳𐏴𐏵𐏶𐏷𐏸𐏹𐏺𐏻𐏼𐏽𐏾𐏿𐐀𐐁𐐂𐐃𐐄𐐅𐐆𐐇𐐈𐐉𐐊𐐋𐐌𐐍𐐎𐐏𐐐𐐑𐐒𐐓𐐔𐐕𐐖𐐗𐐘𐐙𐐚𐐛𐐜𐐝𐐞𐐟𐐠𐐡𐐢𐐣𐐤𐐥𐐦𐐧𐐨𐐩𐐪𐐫𐐬𐐭𐐮𐐯𐐰𐐱𐐲𐐳𐐴𐐵𐐶𐐷𐐸𐐹𐐺𐐻𐐼𐐽𐐾𐐿𐑀𐑁𐑂𐑃𐑄𐑅𐑆𐑇𐑈𐑉𐑊𐑋𐑌𐑍𐑎𐑏𐑐𐑑𐑒𐑓𐑔𐑕𐑖𐑗𐑘𐑙𐑚𐑛𐑜𐑝𐑞𐑟𐑠𐑡𐑢𐑣𐑤𐑥𐑦𐑧𐑨𐑩𐑪𐑫𐑬𐑭𐑮𐑯𐑰𐑱𐑲𐑳𐑴𐑵𐑶𐑷𐑸𐑹𐑺𐑻𐑼𐑽𐑾𐑿𐒀𐒁𐒂𐒃𐒄𐒅𐒆𐒇𐒈𐒉𐒊𐒋𐒌𐒍𐒎𐒏𐒐𐒑𐒒𐒓𐒔𐒕𐒖𐒗𐒘𐒙𐒚𐒛𐒜𐒝𐒞𐒟𐒠𐒡𐒢𐒣𐒤𐒥𐒦𐒧𐒨𐒩𐒪𐒫𐒬𐒭𐒮𐒯𐒰𐒱𐒲𐒳𐒴𐒵𐒶𐒷𐒸𐒹𐒺𐒻𐒼𐒽𐒾𐒿𐓀𐓁𐓂𐓃𐓄𐓅𐓆𐓇𐓈𐓉𐓊𐓋𐓌𐓍𐓎𐓏𐓐𐓑𐓒𐓓𐓔𐓕𐓖𐓗𐓘𐓙𐓚𐓛𐓜𐓝𐓞𐓟𐓠𐓡𐓢𐓣𐓤𐓥𐓦𐓧𐓨𐓩𐓪𐓫𐓬𐓭𐓮𐓯𐓰𐓱𐓲𐓳𐓴𐓵𐓶𐓷𐓸𐓹𐓺𐓻𐓼𐓽𐓾𐓿𐔀𐔁𐔂𐔃𐔄𐔅𐔆𐔇𐔈𐔉𐔊𐔋𐔌𐔍𐔎𐔏𐔐𐔑𐔒𐔓𐔔𐔕𐔖𐔗𐔘𐔙𐔚𐔛𐔜𐔝𐔞𐔟𐔠𐔡𐔢𐔣𐔤𐔥𐔦𐔧𐔨𐔩𐔪𐔫𐔬𐔭𐔮𐔯𐔰𐔱𐔲𐔳𐔴𐔵𐔶𐔷𐔸𐔹𐔺𐔻𐔼𐔽𐔾𐔿𐕀𐕁𐕂𐕃𐕄𐕅𐕆𐕇𐕈𐕉𐕊𐕋𐕌𐕍𐕎𐕏𐕐𐕑𐕒𐕓𐕔𐕕𐕖𐕗𐕘𐕙𐕚𐕛𐕜𐕝𐕞𐕟𐕠𐕡𐕢𐕣𐕤𐕥𐕦𐕧𐕨𐕩𐕪𐕫𐕬𐕭𐕮𐕯𐕰𐕱𐕲𐕳𐕴𐕵𐕶𐕷𐕸𐕹𐕺𐕻𐕼𐕽𐕾𐕿𐖀𐖁𐖂𐖃𐖄𐖅𐖆𐖇𐖈𐖉𐖊𐖋𐖌𐖍𐖎𐖏𐖐𐖑𐖒𐖓𐖔𐖕𐖖𐖗𐖘𐖙𐖚𐖛𐖜𐖝𐖞𐖟𐖠𐖡𐖢𐖣𐖤𐖥𐖦𐖧𐖨𐖩𐖪𐖫𐖬𐖭𐖮𐖯𐖰𐖱𐖲𐖳𐖴𐖵𐖶𐖷𐖸𐖹𐖺𐖻𐖼𐖽𐖾𐖿𐗀𐗁𐗂𐗃𐗄𐗅𐗆𐗇𐗈𐗉𐗊𐗋𐗌𐗍𐗎𐗏𐗐𐗑𐗒𐗓𐗔𐗕𐗖𐗗𐗘𐗙𐗚𐗛𐗜𐗝𐗞𐗟𐗠𐗡𐗢𐗣𐗤𐗥𐗦𐗧𐗨𐗩𐗪𐗫𐗬𐗭𐗮𐗯𐗰𐗱𐗲𐗳𐗴𐗵𐗶𐗷𐗸𐗹𐗺𐗻𐗼𐗽𐗾𐗿𐘀𐘁𐘂𐘃𐘄𐘅𐘆𐘇𐘈𐘉𐘊𐘋𐘌𐘍𐘎𐘏𐘐𐘑𐘒𐘓𐘔𐘕𐘖𐘗𐘘𐘙𐘚𐘛𐘜𐘝𐘞𐘟𐘠𐘡𐘢𐘣𐘤𐘥𐘦𐘧𐘨𐘩𐘪𐘫𐘬𐘭𐘮𐘯𐘰𐘱𐘲𐘳𐘴𐘵𐘶𐘷𐘸𐘹𐘺𐘻𐘼𐘽𐘾𐘿𐙀𐙁𐙂𐙃𐙄𐙅𐙆𐙇𐙈𐙉𐙊𐙋𐙌𐙍𐙎𐙏𐙐𐙑𐙒𐙓𐙔𐙕𐙖𐙗𐙘𐙙𐙚𐙛𐙜𐙝𐙞𐙟𐙠𐙡𐙢𐙣𐙤𐙥𐙦𐙧𐙨𐙩𐙪𐙫𐙬𐙭𐙮𐙯𐙰𐙱𐙲𐙳𐙴𐙵𐙶𐙷𐙸𐙹𐙺𐙻𐙼𐙽𐙾𐙿𐚀𐚁𐚂𐚃𐚄𐚅𐚆𐚇𐚈𐚉𐚊𐚋𐚌𐚍𐚎𐚏𐚐𐚑𐚒𐚓𐚔𐚕𐚖𐚗𐚘𐚙𐚚𐚛𐚜𐚝𐚞𐚟𐚠𐚡𐚢𐚣𐚤𐚥𐚦𐚧𐚨𐚩𐚪𐚫𐚬𐚭𐚮𐚯𐚰𐚱𐚲𐚳𐚴𐚵𐚶𐚷𐚸𐚹𐚺𐚻𐚼𐚽𐚾𐚿𐛀𐛁𐛂𐛃𐛄𐛅𐛆𐛇𐛈𐛉𐛊𐛋𐛌𐛍𐛎𐛏𐛐𐛑𐛒𐛓𐛔𐛕𐛖𐛗𐛘𐛙𐛚𐛛𐛜𐛝𐛞𐛟𐛠𐛡𐛢𐛣𐛤𐛥𐛦𐛧𐛨𐛩𐛪𐛫𐛬𐛭𐛮𐛯𐛰𐛱𐛲𐛳𐛴𐛵𐛶𐛷𐛸𐛹𐛺𐛻𐛼𐛽𐛾𐛿𐜀𐜁𐜂𐜃𐜄𐜅𐜆𐜇𐜈𐜉𐜊𐜋𐜌𐜍𐜎𐜏𐜐𐜑𐜒𐜓𐜔𐜕𐜖𐜗𐜘𐜙𐜚𐜛𐜜𐜝𐜞𐜟𐜠𐜡𐜢𐜣𐜤𐜥𐜦𐜧𐜨𐜩𐜪𐜫𐜬𐜭𐜮𐜯𐜰𐜱𐜲𐜳𐜴𐜵𐜶𐜷𐜸𐜹𐜺𐜻𐜼𐜽𐜾𐜿𐝀𐝁𐝂𐝃𐝄𐝅𐝆𐝇𐝈𐝉𐝊𐝋𐝌𐝍𐝎𐝏𐝐𐝑𐝒𐝓𐝔𐝕𐝖𐝗𐝘𐝙𐝚𐝛𐝜𐝝𐝞𐝟𐝠𐝡𐝢𐝣𐝤𐝥𐝦𐝧𐝨𐝩𐝪𐝫𐝬𐝭𐝮𐝯𐝰𐝱𐝲𐝳𐝴𐝵𐝶𐝷𐝸𐝹𐝺𐝻𐝼𐝽𐝾𐝿𐞀𐞁𐞂𐞃𐞄𐞅𐞆𐞇𐞈𐞉𐞊𐞋𐞌𐞍𐞎𐞏𐞐𐞑𐞒𐞓𐞔𐞕𐞖𐞗𐞘𐞙𐞚𐞛𐞜𐞝𐞞𐞟𐞠𐞡𐞢𐞣𐞤𐞥𐞦𐞧𐞨𐞩𐞪𐞫𐞬𐞭𐞮𐞯𐞰𐞱𐞲𐞳𐞴𐞵𐞶𐞷𐞸𐞹𐞺𐞻𐞼𐞽𐞾𐞿𐟀𐟁𐟂𐟃𐟄𐟅𐟆𐟇𐟈𐟉𐟊𐟋𐟌𐟍𐟎𐟏𐟐𐟑𐟒𐟓𐟔𐟕𐟖𐟗𐟘𐟙𐟚𐟛𐟜𐟝𐟞𐟟𐟠𐟡𐟢𐟣𐟤𐟥𐟦𐟧𐟨𐟩𐟪𐟫𐟬𐟭𐟮𐟯𐟰𐟱𐟲𐟳𐟴𐟵𐟶𐟷𐟸𐟹𐟺𐟻𐟼𐟽𐟾𐟿𐠀𐠁𐠂𐠃𐠄𐠅𐠆𐠇𐠈𐠉𐠊𐠋𐠌𐠍𐠎𐠏𐠐𐠑𐠒𐠓𐠔𐠕𐠖𐠗𐠘𐠙𐠚𐠛𐠜𐠝𐠞𐠟𐠠𐠡𐠢𐠣𐠤𐠥𐠦𐠧𐠨𐠩𐠪𐠫𐠬𐠭𐠮𐠯𐠰𐠱𐠲𐠳𐠴𐠵𐠶𐠷𐠸𐠹𐠺𐠻𐠼𐠽𐠾𐠿𐡀𐡁𐡂𐡃𐡄𐡅𐡆𐡇𐡈𐡉𐡊𐡋𐡌𐡍𐡎𐡏𐡐𐡑𐡒𐡓𐡔𐡕𐡖𐡗𐡘𐡙𐡚𐡛𐡜𐡝𐡞𐡟𐡠𐡡𐡢𐡣𐡤𐡥𐡦𐡧𐡨𐡩𐡪𐡫𐡬𐡭𐡮𐡯𐡰𐡱𐡲𐡳𐡴𐡵𐡶𐡷𐡸𐡹𐡺𐡻𐡼𐡽𐡾𐡿𐢀𐢁𐢂𐢃𐢄𐢅𐢆𐢇𐢈𐢉𐢊𐢋𐢌𐢍𐢎𐢏𐢐𐢑𐢒𐢓𐢔𐢕𐢖𐢗𐢘𐢙𐢚𐢛𐢜𐢝𐢞𐢟𐢠𐢡𐢢𐢣𐢤𐢥𐢦𐢧𐢨𐢩𐢪𐢫𐢬𐢭𐢮𐢯𐢰𐢱𐢲𐢳𐢴𐢵𐢶𐢷𐢸𐢹𐢺𐢻𐢼𐢽𐢾𐢿𐣀𐣁𐣂𐣃𐣄𐣅𐣆𐣇𐣈𐣉𐣊𐣋𐣌𐣍𐣎𐣏𐣐𐣑𐣒𐣓𐣔𐣕𐣖𐣗𐣘𐣙𐣚𐣛𐣜𐣝𐣞𐣟𐣠𐣡𐣢𐣣𐣤𐣥𐣦𐣧𐣨𐣩𐣪𐣫𐣬𐣭𐣮𐣯𐣰𐣱𐣲𐣳𐣴𐣵𐣶𐣷𐣸𐣹𐣺𐣻𐣼𐣽𐣾𐣿𐤀𐤁𐤂𐤃𐤄𐤅𐤆𐤇𐤈𐤉𐤊𐤋𐤌𐤍𐤎𐤏𐤐𐤑𐤒𐤓𐤔𐤕𐤖𐤗𐤘𐤙𐤚𐤛𐤜𐤝𐤞𐤟𐤠𐤡𐤢𐤣𐤤𐤥𐤦𐤧𐤨𐤩𐤪𐤫𐤬𐤭𐤮𐤯𐤰𐤱𐤲𐤳𐤴𐤵𐤶𐤷𐤸𐤹𐤺𐤻𐤼𐤽𐤾𐤿𐥀𐥁𐥂𐥃𐥄𐥅𐥆𐥇𐥈𐥉𐥊𐥋𐥌𐥍𐥎𐥏𐥐𐥑𐥒𐥓𐥔𐥕𐥖𐥗𐥘𐥙𐥚𐥛𐥜𐥝𐥞𐥟𐥠𐥡𐥢𐥣𐥤𐥥𐥦𐥧𐥨𐥩𐥪𐥫𐥬𐥭𐥮𐥯𐥰𐥱𐥲𐥳𐥴𐥵𐥶𐥷𐥸𐥹𐥺𐥻𐥼𐥽𐥾𐥿𐦀𐦁𐦂𐦃𐦄𐦅𐦆𐦇𐦈𐦉𐦊𐦋𐦌𐦍𐦎𐦏𐦐𐦑𐦒𐦓𐦔𐦕𐦖𐦗𐦘𐦙𐦚𐦛𐦜𐦝𐦞𐦟𐦠𐦡𐦢𐦣𐦤𐦥𐦦𐦧𐦨𐦩𐦪𐦫𐦬𐦭𐦮𐦯𐦰𐦱𐦲𐦳𐦴𐦵𐦶𐦷𐦸𐦹𐦺𐦻𐦼𐦽𐦾𐦿𐧀𐧁𐧂𐧃𐧄𐧅𐧆𐧇𐧈𐧉𐧊𐧋𐧌𐧍𐧎𐧏𐧐𐧑𐧒𐧓𐧔𐧕𐧖𐧗𐧘𐧙𐧚𐧛𐧜𐧝𐧞𐧟𐧠𐧡𐧢𐧣𐧤𐧥𐧦𐧧𐧨𐧩𐧪𐧫𐧬𐧭𐧮𐧯𐧰𐧱𐧲𐧳𐧴𐧵𐧶𐧷𐧸𐧹𐧺𐧻𐧼𐧽𐧾𐧿𐨀𐨁𐨂𐨃𐨄𐨅𐨆𐨇𐨈𐨉𐨊𐨋𐨌𐨍𐨎𐨏𐨐𐨑𐨒𐨓𐨔𐨕𐨖𐨗𐨘𐨙𐨚𐨛𐨜𐨝𐨞𐨟𐨠𐨡𐨢𐨣𐨤𐨥𐨦𐨧𐨨𐨩𐨪𐨫𐨬𐨭𐨮𐨯𐨰𐨱𐨲𐨳𐨴𐨵𐨶𐨷𐨹𐨺𐨸𐨻𐨼𐨽𐨾𐨿𐩀𐩁𐩂𐩃𐩄𐩅𐩆𐩇𐩈𐩉𐩊𐩋𐩌𐩍𐩎𐩏𐩐𐩑𐩒𐩓𐩔𐩕𐩖𐩗𐩘𐩙𐩚𐩛𐩜𐩝𐩞𐩟𐩠𐩡𐩢𐩣𐩤𐩥𐩦𐩧𐩨𐩩𐩪𐩫𐩬𐩭𐩮𐩯𐩰𐩱𐩲𐩳𐩴𐩵𐩶𐩷𐩸𐩹𐩺𐩻𐩼𐩽𐩾𐩿𐪀𐪁𐪂𐪃𐪄𐪅𐪆𐪇𐪈𐪉𐪊𐪋𐪌𐪍𐪎𐪏𐪐𐪑𐪒𐪓𐪔𐪕𐪖𐪗𐪘𐪙𐪚𐪛𐪜𐪝𐪞𐪟𐪠𐪡𐪢𐪣𐪤𐪥𐪦𐪧𐪨𐪩𐪪𐪫𐪬𐪭𐪮𐪯𐪰𐪱𐪲𐪳𐪴𐪵𐪶𐪷𐪸𐪹𐪺𐪻𐪼𐪽𐪾𐪿𐫀𐫁𐫂𐫃𐫄𐫅𐫆𐫇𐫈𐫉𐫊𐫋𐫌𐫍𐫎𐫏𐫐𐫑𐫒𐫓𐫔𐫕𐫖𐫗𐫘𐫙𐫚𐫛𐫜𐫝𐫞𐫟𐫠𐫡𐫢𐫣𐫤𐫦𐫥𐫧𐫨𐫩𐫪𐫫𐫬𐫭𐫮𐫯𐫰𐫱𐫲𐫳𐫴𐫵𐫶𐫷𐫸𐫹𐫺𐫻𐫼𐫽𐫾𐫿𐬀𐬁𐬂𐬃𐬄𐬅𐬆𐬇𐬈𐬉𐬊𐬋𐬌𐬍𐬎𐬏𐬐𐬑𐬒𐬓𐬔𐬕𐬖𐬗𐬘𐬙𐬚𐬛𐬜𐬝𐬞𐬟𐬠𐬡𐬢𐬣𐬤𐬥𐬦𐬧𐬨𐬩𐬪𐬫𐬬𐬭𐬮𐬯𐬰𐬱𐬲𐬳𐬴𐬵𐬶𐬷𐬸𐬹𐬺𐬻𐬼𐬽𐬾𐬿𐭀𐭁𐭂𐭃𐭄𐭅𐭆𐭇𐭈𐭉𐭊𐭋𐭌𐭍𐭎𐭏𐭐𐭑𐭒𐭓𐭔𐭕𐭖𐭗𐭘𐭙𐭚𐭛𐭜𐭝𐭞𐭟𐭠𐭡𐭢𐭣𐭤𐭥𐭦𐭧

§ 1. Introduction

A new I/O-agnostic text formatting library was introduced in C++20 ([\[FORMAT\]](#)). This paper proposes integrating it with standard I/O facilities via a simple and intuitive API achieving the following goals:

- Usability
- Unicode support
- Good performance
- Small binary footprint

§ 2. Revision history

Changes since R10:

- Moved "If invoking the (native Unicode) API requires transcoding, implementations should substitute invalid code units with U+FFFD REPLACEMENT CHARACTER per The Unicode Standard Version 13.0 – Core Specification, Chapter 3.9." to the *Recommended practice* clause in the definition of `vprint_unicode` per LEWG feedback.
- Made the definition of the `ostream` overload of `vprint_unicode` consistent with the `FILE*` overload.
- Namespace-qualified the calls to `forward`.
- Removed a call to `make_format_args` in `[print.fun].1`.
- Removed extra `)` in the *Effects* clauses of `vprint_unicode` and `vprint_nonunicode`.
- Rendered `str` in italics in `fmt.str`.
- Defined the `println` overload that doesn't take a stream in terms of the one that does.
- Add missing `.str` to `fmt` in the `ostream` overload of `print`.
- Updated The Unicode® Standard Version to 14.

Changes since R9:

- Rebased the wording onto the latest draft, most importantly adding support for types that are neither const-iterable nor copyable introduced by [\[P2418\]](#).

Changes since R8:

- Added new SG16 poll results.
- Improved wording for `[print.syn].6` (previously `[print.syn].31`) to remove ambiguities per SG16 feedback.
- Fixed paragraph numbering in the wording.
- Clarified the difference from the Python's `print` in [§ 10 API and naming](#).

Changes since R7:

- Added a reference to LLVM's `raw_ostream` that implements similar mojibake prevention mechanism to [§ 15 Implementation](#).

Changes since R6:

- Added new SG16 poll results.
- Rebased the wording onto the latest draft, most importantly adding compile-time checks introduced by [\[P2216\]](#).
- Added "If `out` contains invalid code units, the behavior is undefined and implementations are encouraged to diagnose it." to the definition of `vprint_unicode` per SG16 feedback.
- Replaced "invalid code points are substituted with U+FFFD  REPLACEMENT CHARACTER" with "implementations should substitute invalid code units with U+FFFD  REPLACEMENT CHARACTER per The Unicode® Standard Version 13.0 – Core Specification, Chapter 3.9" in [§ 16 Wording](#) per SG16 feedback.
- Added "The Unicode® Standard Version 13.0 – Core Specification" to Normative references in [§ 16 Wording](#).
- Clarified the behavior when mixing encodings in [§ 11 Unicode](#).

Changes since R5:

- Added new LEWG poll results.
- Added new SG16 poll results.
- Replaced `<io>` with `<print>` per LEWG feedback.
- Clarified the choice of U+FFFD  REPLACEMENT CHARACTER for transcoding errors in [§ 11 Unicode](#).
- Clarified the choice of literal encoding in [§ 11 Unicode](#).
- Clarified that ANSI escape codes for specifying coding systems are not considered a native system API that supports Unicode in [§ 11 Unicode](#).
- Added a reference to Rust's standard output facility that implements the same mojibake prevention mechanism to [§ 15 Implementation](#)

Changes since R4:

- Added SG16 Unicode poll results.
- Added a list of candidate headers formatted output functions can be added to.
- Moved the non-`ostream` overloads of formatted output functions to a separate header for cleaner separation of formatting and I/O facilities.

Changes since R3:

- Replaced `_isatty(_fileno(stream))` with `GetConsoleMode(_get_osfhandle(_fileno(stream)), ...)` in a note in `[format.functions]` because the former may return 1 for streams not referring to a terminal.

Changes since R2:

- Added better compatibility with other formatted I/O facilities as another advantage of using `stdout` in §10 [API and naming](#) per SG16 feedback.
- Clarified that [\[P1885\]](#) can be used for literal encoding detection per SG16 feedback.
- Added comparison of Unicode handling in various languages in [Appendix A: Unicode tests](#) and a summary in [§11 Unicode](#) per SG16 request.
- Removed incorrect "exposition-only" in [§11 Unicode](#).
- Replaced "both source and literal encodings are UTF-8, which is enabled by the `/utf-8` compiler flag" with "the literal (execution) encoding is UTF-8, which is enabled by the `/execution-charset:utf-8` compiler flag" since the source encoding is irrelevant there.
- Rephrased *Effects* of `vprint_unicode` for clarity.

Changes since R1:

- Added missing `println` overloads that take `FILE*` and `ostream&`.
- Moved the print functions that take `ostream&` to `<ostream>` since `<format>` shouldn't depend on `<ostream>`.
- Clarified why it is useful to provide `vprint*` functions in [§13 Binary code](#).
- Rebased the wording onto the latest working draft, N4861, in particular updating the *Throws* clauses to match existing wording.
- Replaced `std::system_error` with `system_error` in the wording.
- Added paragraph numbers to the wording.

Changes since R0:

- Clarified that adding `wchar_t` overloads will be addressed in a separate paper per the UK C++ panel feedback.

§ 3. SG16 polls (R8)

Poll: Use of UTF-8 as the literal encoding is sufficient for `<print>` facilities to establish encoding expectations.

SF	F	N	A	SA
2	3	2	1	0

Consensus: Consensus in favour.

Poll: Correct the P2093R8 wording for `[print.syn].31` to remove ambiguities, and forward P2093 as revised to LEWG with a recommended ship vehicle of C++23.

SF	F	N	A	SA
1	4	2	0	0

Consensus: Consensus in favour.

§ 4. SG16 polls (R6)

Poll: When `<print>` facilities must transcode their formatting results for display on a device, and during that process invalidly-encoded text is encountered, `print()` should replace the erroneously-encoded code units with `U+FFFD REPLACEMENT CHARACTER`.

SF	F	N	A	SA
3	3	1	2	0

Outcome: Consensus for the position

Poll: When `<print>` facilities need not transcode their formatting results for display on a device, and during that process invalidly-encoded text is encountered, `print()` should nevertheless replace the erroneously-encoded code units with `U+FFFD REPLACEMENT CHARACTER`.

SF	F	N	A	SA
1	0	2	2	3

Outcome: Consensus against the direction.

Poll: `<format>` and `<print>` facilities should have consistent behavior with respect to encoding expectations for the format string.

No objection to unanimous consent.

Poll: formatters should not be sensitive to whether they are being used with a `<format>` or `<print>` facility.

No objection to unanimous consent.

Poll: Regardless of format string encoding assumptions, `<format>` facilities may be used to format binary data.

SF	F	N	A	SA
5	1	1	0	0

Consensus: Strong consensus in favor.

Poll: Regardless of format string encoding assumptions, `<print>` facilities may be used to format binary data.

SF	F	N	A	SA
2	1	3	1	0

Consensus: Weak consensus in favor.

Poll: `<print>` facilities exhibit undefined behavior when an encoding expectation is present and a format string or formatter output does not match those expectations.

SF	F	N	A	SA
2	4	0	0	1

Consensus: Strong consensus in favor.

Poll: `<print>` facilities exhibit undefined behavior when an encoding expectation is present and a format string or formatter output does not match those expectations and output is directed to a device that has encoding expectations.

SF	F	N	A	SA
6	0	1	0	0

Consensus: Stronger consensus in favor relative to previous poll.

Poll: `<print>` facility implementors are encouraged to provide a run-time means for diagnosing format strings and formatter output that is not well-formed according to the expected encoding.

SF	F	N	A	SA
4	0	2	1	0

Consensus: Consensus in favor.

Poll: `<format>` and `<print>` facilities should have consistent behavior with respect to encoding expectations for the output of formatters.

SF	F	N	A	SA
0	1	1	5	1

Consensus: Strong consensus against.

Poll: `<print>` facility implementors are encouraged to substitute `U+FFFD` replacement characters following Unicode guidance when output is directed to a device and transcoding is necessary.

SF	F	N	A	SA
2	5	0	0	1

Consensus: Consensus in favor.

Poll: `<print>` facilities must provide an explicit program-controlled error handling mechanism for violations of encoding expectations.

SF	F	N	A	SA
0	0	3	3	3

Poll: Use of UTF-8 as the literal encoding is sufficient for `<print>` facilities to establish encoding expectations.

SF	F	N	A	SA
3	1	3	2	0

Consensus: Very weak consensus.

§ 5. LEWG polls (R5)

Poll: Block P2093 until we have a proposal for a lower level facility that can query tty/console and perform direct output to the console

SF	F	N	A	SA
2	1	5	5	6

Outcome: Consensus Against

Poll: We want `std::print` et al. non `ostream` overloads to go to:

Header	Approve	Disapprove
<code><ostream></code>	3	7
<code><io></code>	6	5
<code><print></code>	10	3
<code><format></code>	12	1
<code><utility></code>	1	13

Outcome: both `<print>` and `<format>` have consensus, author can choose one for the next revision.

§ 6. SG16 polls (R3)

Poll: Forward P2093R3 to LEWG.

SF	F	N	A	SA
4	2	2	0	1

Consensus? Yes

§ 7. LEWG polls (R2)

Poll: We want P2093R2 to revert moving the `print(ostream)` overloads to the `<ostream>` header.

SF	F	N	A	SA
0	0	6	6	1

Outcome: Keep the paper as is

§ 8. LEWG polls (R1)

Poll: We prefer `std::cout` as the default output target.

SF	F	N	A	SA
3	2	6	6	5

Poll: Add a member function on `ostream` instead of a `std::print(ostream&, ...)` free function overload.

SF	F	N	A	SA
0	2	5	17	3

Consensus against.

Poll: Remove `std::println` from the paper?

SF	F	N	A	SA
1	10	7	4	4

No consensus for change.

Poll: We are happy with the design with regards to UTF-8 output.

Unanimous consent.

Attendance: 35

§ 9. Motivating examples

Consider a common task of printing formatted text to `stdout`:

C++20	Proposed
<pre>std::cout << std::format("Hello, {}!", name);</pre>	<pre>std::print("Hello, {}!", name);</pre>

The proposed `std::print` function improves usability, avoids allocating a temporary `std::string` object and calling `operator<<` which performs formatted I/O on text that is already formatted. The number of function calls is reduced to one which, together with `std::vformat`-like type erasure, results in much smaller binary code (see [§ 13 Binary code](#)).

Existing alternatives in C++20:

§ 10. API and naming

Many programming languages provide functions for printing text to standard output, often combined with formatting:

Language	Function(s)
C	<code>printf</code> [N2176]
C#/NET	<code>Console.Write</code> [DOTNET-WRITE]
COBOL	<code>DISPLAY</code> statement [N0147]
Fortran	<code>print</code> and <code>write</code> statements [N2162]
Go	<code>Printf</code> [GO-FMT]
Java	<code>PrintStream.format</code> , <code>PrintStream.print</code> , <code>PrintStream.printf</code> [JAVA-PRINT]
JavaScript	<code>console.log</code> [WHATWG-CONSOLE]
Perl	<code>printf</code> [PERL-PRINTF]
PHP	<code>printf</code> [PHP-PRINTF]
Python	<code>print</code> statement or function [PY-FUNC]
R	<code>print</code> [R-PRINT]
Ruby	<code>print</code> and <code>printf</code> [RUBY-PRINT]
Rust	<code>print!</code> [RUST-PRINT]
Swift	<code>print</code> [SWIFT-PRINT]

Variations of `print[f]` appear to be the most popular naming choice for this functionality. It is either provided as a free function (most common) or a member function (less common) together with a global object representing standard output stream. Notable exceptions are COBOL, Fortran, and Python 2 which have dedicated language statements and Rust where `print!` is a function-like macro.

We propose adding a free function called `print` with overloads for writing to the standard output (the default) and an explicitly passed output stream object. The default output stream can be either `stdout` or `std::cout`. We propose using `stdout` for the following reasons:

- `stdout` is considerably faster on at least two major implementations (see § 12 Performance).
- Better compatibility with other formatted I/O facilities compared to `std::cout` and its associated `std::streambuf` that suffer from private buffering, localization and conversion services that must be synchronized at a lower level.
- `print` won't use any formatted output functionality of `ostream`.

In some languages like Python `print` only provides the default formatting although some formatting control may be achieved by other means such as named arguments and/or doing formatting manually via `str.format` or string interpolation. The current paper doesn't propose such a default formatting facility. A search in a large Python codebase revealed that over 70% of `print` calls either take a string literal or use interpolation and this doesn't even account for string variables. These use cases are covered by the current proposal with better usability and potentially better performance because there are no separate formatting function calls.

Since `stdout` doesn't have an associated locale we propose using the current global locale for locale-specific formatting which is consistent with `format`. With `cout` or another explicitly passed stream, the stream's locale will be used. In all cases the default formatting is locale-independent.

Another option is to make `print` a member function of `basic_ostream`. This would make usage somewhat more awkward:

```
std::cout.print("Hello, {}!", name);
```

A free function can also be overloaded to take `FILE*` to simplify migration (possibly automated) of code from `printf` to the new facility.

There are multiple approaches to appending a trailing newline:

- Don't append a newline automatically: `printf` in C and other languages.
- Append a newline but don't format arguments: `puts` in C (inconsistent with `fputs`).
- Have two formatting functions/macros, one that appends newline and another that doesn't: `print/println` in Java, `print!/println!` in Rust, `Printf/Println` in Go, `Write/WriteLine` in C#/.NET.
- Let the user choose a terminating string defaulting to `"\n"` and do limited formatting: `print` in Python and Swift.

We propose not appending a newline automatically for consistency with `printf` and iostreams:

```
std::print("Hello, {}!", name);    // doesn't print a newline
std::print("Hello, {}!\n", name);  // prints a newline
```

Additionally we can provide a function that appends a newline:

```
std::println("Hello, {}!", name);  // prints a newline
```

Although `println` doesn't provide much usability improvement compared to `print` with explicit `'\n'`, it has been an occasionally requested feature in the `fmt` library ([\[FMT\]](#)).

Another question is which header non-`ostream` overloads of formatted output functions should go to. Possible options:

- `<io>`
- `<print>`
- `<format>`
- `<ostream>`
- `<utility>`

Earlier versions of the paper proposed `<io>` analogous to `<stdio>` so that the future I/O facilities that don't depend on ostream could be added there. This was changed to a more narrow-focused `<print>` but `<io>` can be added in the future once a symmetric input facility becomes available. Using `<ostream>` is undesirable because this header and its transitive dependencies are very big (42 thousand lines preprocessed on libc++):

```
% echo '#include <ostream>' | clang++ -E -x c++ - | wc -l
42491
```

It also pulls in a lot of unrelated symbols such as ostream insertion operators, global `cout`, `cerr`, `clog` variables and their `wchar_t` counterparts.

ostream overloads are added to the `<ostream>` header.

§ 11. Unicode

We can prevent mojibake in the Unicode example by detecting if the string literal encoding is UTF-8 and dispatching to a different function that correctly handles Unicode, for example:

```
constexpr bool is_utf8() {
    const unsigned char micro[] = "\u00B5";
    return sizeof(micro) == 3 && micro[0] == 0xC2 && micro[1] == 0xB5;
}

template <typename... Args>
void print(string_view fmt, const Args&... args) {
    if (is_utf8())
        vprint_unicode(fmt, make_format_args(args...));
    else
        vprint_nonunicode(fmt, make_format_args(args...));
}
```

where the `vprint_unicode` function formats and prints text in UTF-8 using the native system API that supports Unicode and `vprint_nonunicode` does the same for other encodings. The latter ensures that interoperability with code using legacy encodings is preserved even though `print` is a new API and it is not strictly necessary. If calling the system API requires transcoding we propose substituting invalid code units with U+FFFD  REPLACEMENT CHARACTER which is consistent with the treatment of malformed UTF-8 in UTF-8-native terminals. For example

```
#include <stdio.h>

int main() {
    puts("\xc3\x28"); // Invalid 2 Octet Sequence
}
```

prints `0(` in iTerm2 and `?(` in macOS Terminal. So whether transcoding is done or not in the UTF-8 case, you will normally get similar observed behavior.

In Visual C++ `is_utf8` will return `true` if the literal (execution) encoding is UTF-8, which is enabled by the `/execution-charset:utf-8` compiler flags or other means, and `false` otherwise. Literal encoding detection can be implemented in a more elegant way using [\[P1885\]](#).

Note that ANSI escape codes for specifying coding systems ([\[ISO2022\]](#)) are not considered a native system API that supports Unicode for the purposes of this proposal.

We propose using the literal encoding for the following reasons:

1. Consistency with the design of `std::format` which is locale-independent by default ([\[P0645\]](#)) and disallows implicitly mixing encodings e.g. passing a narrow string into a wide `std::format` is ill-formed.
2. Consistency with the encoding used for width estimation ([\[P1868\]](#)). The standard wording doesn't mention the literal encoding explicitly but the fact that the format strings are either literals or other compile-time strings ([\[P2216\]](#)) makes it the only conformant option.
3. Safety: the result of `formatted_size` does not depend on the global locale by default and a buffer allocated with this size can be passed safely to `format_to` even if the locale has been changed in the meantime, possibly from another thread.
4. Implementation and usage experience.
5. In the vast majority of cases format strings are literals. For example, analyzing a sample of 100 `printf` calls from [\[CODESEARCH\]](#) showed that 98 of them are string literals and 2 are string literals wrapped in the `__gettext` macro.
6. The active code page and the terminal encoding being unrelated on popular Windows localizations such as Russian where the former is CP1251 while the latter is CP866. Instead of assuming one encoding regardless of the string origin which would often result in mojibake, an explicit encoding indication can be done via the standard extension API, e.g. (exposition only)

```
print("Привет, {}!", locale_enc(string_in_locale_encoding));
```

This is already possible to implement by providing appropriate `std::formatter` specializations.

This approach has been implemented in the `fmt` library ([\[FMT\]](#)), successfully tested and used on a variety of platforms.

Users can sometimes restrict the set of used characters to the common subset among multiple encodings (often ASCII) in which case encoding becomes mostly irrelevant. Such "polyglot" strings are fully supported for legacy encodings and partially supported for UTF-8 by the current proposal even though mixing encodings in such a way is a clearly bad practice from a general software engineering point of view.

Here's an example output on Windows:

```

C:\projects\fmt>cl print-test.cc /I include /utf-8 /EHsc
Microsoft (R) C/C++ Optimizing Compiler Version 19.24.28316 for x86
Copyright (C) Microsoft Corporation. All rights reserved.

print-test.cc
Microsoft (R) Incremental Linker Version 14.24.28316.0
Copyright (C) Microsoft Corporation. All rights reserved.

/out:print-test.exe
print-test.obj

C:\projects\fmt>print-test
Привет, κόσμος!

```

At the same time interoperability with legacy code is preserved when literal encoding is not UTF-8. In particular, in case of EBCDIC, Shift JIS or a non-Unicode Windows code page, `print` will perform no transcoding and the text will be printed as is.

The following table summarizes the behavior of formatted output facilities in different programming languages:

Language	Linux		macOS		Windows	
	Terminal	Redirect	Terminal	Redirect	Terminal	Redirect
C	Correct	UTF-8	Correct	UTF-8	Wrong	UTF-8
Go	Correct	UTF-8	Correct	UTF-8	Correct	UTF-8
Java	Correct	UTF-8*	Correct	UTF-8*	Wrong	CP1251 (lossy)
JavaScript	Correct	UTF-8*	Correct	UTF-8*	Correct	UTF-8*
Python	Correct	UTF-8*	Correct	UTF-8*	Correct	Error
Rust	Correct	UTF-8	Correct	UTF-8	Correct	UTF-8

* - the output is transcoded from a different UTF representation.

Correct means that the test message "Привет, κόσμος!" was fully readable in the terminal output. None of the tested language facilities were able to produce readable output when piped through the standard `findstr` command on Windows. Java gave the worst results producing both mojibake and replacement characters in this case: "␣Ειμήτορ, ??????". Most other languages produced valid UTF-8 when the output of `findstr` was redirected to a file.

The current paper proposes following C, Go, JavaScript and Rust and preserving the original encoding (modulo UTF conversion). The only difference compared to `printf` is that we fix the console output on Windows. Java's approach is problematic for the following reasons:

- There is a silent data loss for **valid** Unicode code points when the output is redirected to a file.
- It is more expensive because of transcoding.
- It may give an unusable result when piped through standard Windows commands like `findstr`.

- It transcodes into legacy encodings that are rarely used in practice nowadays. For example, usage of CP1251 dropped from 4.3% to 0.6% in the last 12+ years ([\[ENCODING-TRENDS\]](#)), including a 0.4% drop while the current paper was in review.

The full listings of test programs are given in [Appendix A: Unicode tests](#).

§ 12. Performance

All the performance benefits of `std::format` ([\[FORMAT\]](#)) automatically carry over to this proposal. In particular, locale-independence by default reduces global state and makes formatting more efficient compared to `stdio` and `iostreams`. There are fewer function calls (see [§ 13 Binary code](#)) and no shared formatting state compared to `iostreams`.

The following benchmark compares the reference implementation of `print` with `printf` and `ostream`. This benchmark formats a simple message and prints it to the output stream redirected to `/dev/null`. It uses the Google Benchmark library [\[GOOGLE-BENCH\]](#) to measure timings:

```
#include <cstdio>
#include <iostream>

#include <benchmark/benchmark.h>
#include <fmt/ostream.h>

void printf(benchmark::State& s) {
    while (s.KeepRunning())
        std::printf("The answer is %d.\n", 42);
}
BENCHMARK(printf);

void ostream(benchmark::State& s) {
    std::ios::sync_with_stdio(false);
    while (s.KeepRunning())
        std::cout << "The answer is " << 42 << ".\n";
}
BENCHMARK(ostream);

void print(benchmark::State& s) {
    while (s.KeepRunning())
        fmt::print("The answer is {}.\n", 42);
}
BENCHMARK(print);

void print_cout(benchmark::State& s) {
    std::ios::sync_with_stdio(false);
    while (s.KeepRunning())
```



```

    fmt::print(std::cout, "The answer is {}.\\n", 42);
}
BENCHMARK(print_cout);

void print_cout_sync(benchmark::State& s) {
    std::ios::sync_with_stdio(true);
    while (s.KeepRunning())
        fmt::print(std::cout, "The answer is {}.\\n", 42);
}
BENCHMARK(print_cout_sync);

BENCHMARK_MAIN();

```

The benchmark was compiled with Apple clang version 11.0.0 (clang-1100.0.33.17) with `-O3 -DNDEBUG` and run on macOS 10.15.4. Below are the results:

Run on (8 X 2800 MHz CPU s)

CPU Caches:

L1 Data 32K (x4)

L1 Instruction 32K (x4)

L2 Unified 262K (x4)

L3 Unified 8388K (x1)

Load Average: 1.83, 1.88, 1.82

Benchmark	Time	CPU	Iterations
printf	87.0 ns	86.9 ns	7834009
ostream	255 ns	255 ns	2746434
print	78.4 ns	78.3 ns	9095989
print_cout	89.4 ns	89.4 ns	7702973
print_cout_sync	91.5 ns	91.4 ns	7903889

Both `print` and `printf` are ~3 times faster than `cout` even with synchronization to the standard C streams turned off. `print` is 14% faster when printing to `stdout` than to `cout`. For this reason and because `print` doesn't use formatting facilities of `ostream` we propose using `stdout` as the default output stream and providing an overload for writing to `ostream`.

On Windows 10 with Visual C++ 2019 the results are similar although the difference between `print` writing to `stdout` and `cout` is smaller with `stdout` being 7% faster:

Run on (1 X 2808 MHz CPU)

CPU Caches:

L1 Data 32K (x1)

L1 Instruction 32K (x1)

L2 Unified 262K (x1)

L3 Unified 8388K (x1)

Benchmark	Time	CPU	Iterations
printf	835 ns	816 ns	746667
ostream	2410 ns	2400 ns	280000
print	580 ns	572 ns	1120000
print_cout	623 ns	614 ns	1120000
print_cout_sync	615 ns	614 ns	1120000

§ 13. Binary code

We propose minimizing per-call binary code size by applying the type erasure mechanism from [\[P0645\]](#). In this approach all the formatting and printing logic is implemented in a non-variadic function `vprint`. Inline variadic `print` function only constructs a `format_args` object, representing an array of type-erased argument references, and passes it to `vprint*`. Here is a simplified example:

```
void vprint(string_view fmt, format_args args);

template<class... Args>
inline void print(string_view fmt, const Args&... args) {
    return vprint(fmt, make_format_args(args...));
}
```

We provide `vprint*` overloads so that users can apply the same technique to their own code. For example:

```
void vlog(log_level level, string_view fmt, format_args args) {
    // Print the log level and use vprint* overloads to format and print the
    // message.
}

template<class... Args>
inline void log(log_level level, string_view fmt, const Args&... args) {
    return vlog(level, fmt, make_format_args(args...));
}
```

Here `vlog` that implements the logging logic is not parameterized on formatting argument types resulting in less code bloat compared to a naive templated version. As a real-world example, this technique has been applied in the Folly Logger ([\[FOLLY\]](#)) bringing ~5x binary size reduction per logging function call.

Below we compare the reference implementation of `print` to standard formatting facilities. All the code snippets are compiled with clang (Apple clang version 11.0.0 clang-1100.0.33.17) with `-O3 -DNDEBUG -c -std=c++17` and the resulting binaries are disassembled with `objdump -S`:

```
void printf_test(const char* name) {  
    printf("Hello, %s!", name);  
}
```

```
__Z11printf_testPKc:  
0:      55      pushq   %rbp  
1:      48 89 e5      movq    %rsp, %rbp  
4:      48 89 fe      movq    %rdi, %rsi  
7:      48 8d 3d 08 00 00 00    leaq    8(%rip), %rdi  
e:      31 c0      xorl    %eax, %eax  
10:     5d      popq    %rbp  
11:     e9 00 00 00 00    jmp     0 <__Z11printf_testPKc+0x16>
```

```
void ostream_test(const char* name) {  
    std::cout << "Hello, " << name << "!";  
}
```

```
__Z12ostream_testPKc:  
0:      55      pushq   %rbp  
1:      48 89 e5      movq    %rsp, %rbp  
4:      41 56      pushq   %r14  
6:      53      pushq   %rbx  
7:      48 89 fb      movq    %rdi, %rbx  
a:      48 8b 3d 00 00 00 00    movq    (%rip), %rdi  
11:     48 8d 35 6c 03 00 00    leaq    876(%rip), %rsi  
18:     ba 07 00 00 00    movl    $7, %edx  
1d:     e8 00 00 00 00    callq   0 <__Z12ostream_testPKc+0x22>  
22:     49 89 c6      movq    %rax, %r14  
25:     48 89 df      movq    %rbx, %rdi  
28:     e8 00 00 00 00    callq   0 <__Z12ostream_testPKc+0x2d>  
2d:     4c 89 f7      movq    %r14, %rdi  
30:     48 89 de      movq    %rbx, %rsi  
33:     48 89 c2      movq    %rax, %rdx  
36:     e8 00 00 00 00    callq   0 <__Z12ostream_testPKc+0x3b>  
3b:     48 8d 35 4a 03 00 00    leaq    842(%rip), %rsi  
42:     ba 01 00 00 00    movl    $1, %edx  
47:     48 89 c7      movq    %rax, %rdi  
4a:     5b      popq    %rbx  
4b:     41 5e      popq    %r14  
4d:     5d      popq    %rbp
```

```

4e:      e9 00 00 00 00  jmp      0 <__Z12ostream_testPKc+0x53>
53:      66 2e 0f 1f 84 00 00 00 00 00  nopw    %cs:(%rax,%rax)
5d:      0f 1f 00          nopl     (%rax)

```

```

void print_test(const char* name) {
    print("Hello, {}!", name);
}

```

```

__Z10print_testPKc:
    0:      55          pushq   %rbp
    1:      48 89 e5          movq    %rsp, %rbp
    4:      48 83 ec 10       subq    $16, %rsp
    8:      48 89 7d f0       movq    %rdi, -16(%rbp)
    c:      48 8d 3d 19 00 00 00 leaq    25(%rip), %rdi
   13:      48 8d 4d f0       leaq    -16(%rbp), %rcx
   17:      be 0a 00 00 00     movl    $10, %esi
   1c:      ba 0d 00 00 00     movl    $13, %edx
   21:      e8 00 00 00 00     callq   0 <__Z10print_testPKc+0x26>
   26:      48 83 c4 10       addq    $16, %rsp
   2a:      5d          popq    %rbp
   2b:      c3          retq

```

The code generated for the `print_test` function that uses the reference implementation of `print` described in this proposal is more than 2x smaller than the ostream code and has one function call instead of three. The `printf` code is further 2x smaller but doesn't have any error handling. Adding error handling would make its code size closer to that of `print`.

The following factors contribute to the difference in binary code size between `print` and `printf`:

- Passing format string as `string_view` instead of `const char*`.
- Capturing and passing argument type information.
- Preparing the array of formatting arguments.

§ 14. Impact on existing code

The current proposal adds new functions to the headers `<print>` and `<ostream>` and should have no impact on existing code.

§ 15. Implementation

The proposed `print` function has been implemented in the open-source fmt library [\[FMT\]](#) and has been in use for about 6 years.

Rust's standard output facility uses essentially the same approach for preventing mojibake when printing to console on Windows ([[RUST-STDIO](#)]). The main difference is that invalid code units are reported as errors in Rust.

LLVM's [raw_ostream](#) [[LLVM-OSTREAM](#)] also implements this approach when writing to console on Windows. The main difference is that in case of invalid UTF-8 it falls back on writing raw (not transcoded) data.

§ 16. Wording

Add an entry for `__cpp_lib_print` to section "Header `<version>` synopsis [[version.syn](#)]", in a place that respects the table's current alphabetic order:

```
#define __cpp_lib_print 202111L **placeholder** // also in <format>
```

Add the header `<print>` to the "C++ library headers" table in [headers], in a place that respects the table's current alphabetic order.

29.7.? Header `<print>` synopsis [[print.syn](#)].

```
namespace std {  
    template<class... Args>  
        void print(format_string<Args...> fmt, Args&&... args);  
    template<class... Args>  
        void print(FILE* stream, format_string<Args...> fmt, Args&&... args);  
  
    template<class... Args>  
        void println(format_string<Args...> fmt, Args&&... args);  
    template<class... Args>  
        void println(FILE* stream, format_string<Args...> fmt, Args&&... args);  
  
    void vprint_unicode(string_view fmt, format_args args);  
    void vprint_unicode(FILE* stream, string_view fmt, format_args args);  
  
    void vprint_nonunicode(string_view fmt, format_args args);  
    void vprint_nonunicode(FILE* stream, string_view fmt, format_args args);  
}
```

Modify section "Header `<ostream>` synopsis [[ostream.syn](#)]":

```
...  
template<class charT, class traits, class T>  
basic_ostream<charT, traits>& operator<< (basic_ostream<charT, traits>&& os, const T  
  
template<class... Args>
```

```

    void print(ostream& os, format-string<Args...> fmt, Args&&... args);
template<class... Args>
    void println(ostream& os, format-string<Args...> fmt, Args&&... args);

void vprint_unicode(ostream& os, string_view fmt, format_args args);
void vprint_nonunicode(ostream& os, string_view fmt, format_args args);

```

29.7.7 Print functions [print.fun].

```

template<class... Args>
    void print(format-string<Args...> fmt, Args&&... args);

```

1 *Effects:* Equivalent to:

```

    print(stdout, fmt, std::forward<Args>(args)...);

```

```

template<class... Args>
    void print(FILE* stream, format-string<Args...> fmt, Args&&... args);

```

2 *Effects:* If string literal encoding is UTF-8, equivalent to:

```

    vprint_unicode(stream, fmt.str, make_format_args(std::forward<Args>(args)...));

```

Otherwise, equivalent to:

```

    vprint_nonunicode(stream, fmt.str, make_format_args(std::forward<Args>(args)...))

```

```

template<class... Args>
    void println(format-string<Args...> fmt, Args&&... args);

```

3 *Effects:* Equivalent to:

```

    println(stdout, fmt, std::forward(args)...);

```

```

template<class... Args>
    void println(FILE* stream, format-string<Args...> fmt, Args&&... args);

```

Effects: Equivalent to:

```
print(stream, "{}\n", format(fmt, std::forward<Args>(args)...));
```

```
void vprint_unicode(string_view fmt, format_args args);
```

5 *Effects:* Equivalent to:

```
vprint_unicode(stdout, fmt, args);
```

```
void vprint_unicode(FILE* stream, string_view fmt, format_args args);
```

6 *Effects:* Let `out = vformat(fmt, args)`. If `stream` refers to a terminal capable of displaying Unicode, writes `out` to the terminal using the native Unicode API; if `out` contains invalid code units, the behavior is undefined and implementations are encouraged to diagnose it. Otherwise writes `out` to `stream` unchanged.

[Note: On POSIX and Windows, `stream` referring to a terminal means that, respectively, `isatty(fileno(stream))` and `GetConsoleMode(_get_osfhandle(_fileno(stream)), ...)` return nonzero. — end note]

[Note: On Windows, the native Unicode API is `WriteConsoleW`. — end note]

7 *Throws:* As specified in `[format.err.report]` or `system_error` if a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications.

8 *Recommended practice:* If invoking the native Unicode API requires transcoding, implementations should substitute invalid code units with U+FFFD REPLACEMENT CHARACTER per The Unicode Standard Version 14.0 – Core Specification, Chapter 3.9.

```
void vprint_nonunicode(string_view fmt, format_args args);
```

9 *Effects:* Equivalent to:

```
vprint_nonunicode(stdout, fmt, args);
```

```
void vprint_nonunicode(FILE* stream, string_view fmt, format_args args);
```

10 *Effects:* Writes the result of `vformat(fmt, args)` to `stream`.

11 *Throws:* As specified in `[format.err.report]` or `system_error` if a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications.

Add subsection "Print [ostream.formatted.print]" to "Formatted output functions [ostream.formatted]":

```
template<class... Args>
void print(ostream& os, format-string<Args...> fmt, Args&&... args);
```

- 1 *Effects:* If string literal encoding is UTF-8, equivalent to:

```
vprint_unicode(os, fmt.str, make_format_args(std::forward<Args>(args)...));
```

Otherwise, equivalent to:

```
vprint_nonunicode(os, fmt.str, make_format_args(std::forward<Args>(args)...));
```

```
void vprint_unicode(ostream& os, string_view fmt, format_args args);
```

- 2 *Effects:* Let `out = vformat(os.getloc(), fmt, args)`. If `os` is a stream that refers to a terminal capable of displaying Unicode which is determined in an implementation-defined manner, writes `out` to the terminal using the native Unicode API; if `out` contains invalid code units, the behavior is undefined and implementations are encouraged to diagnose it. Otherwise writes `out` to `stream` unchanged.

Throws: As specified in [format.err.report] or `system_error` if a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications.

- 3 *Recommended practice:* If invoking the native Unicode API requires transcoding, implementations should substitute invalid code units with U+FFFD REPLACEMENT CHARACTER per The Unicode Standard Version 14.0 – Core Specification, Chapter 3.9.

```
void vprint_nonunicode(ostream& os, string_view fmt, format_args args);
```

- 4 *Effects:* Writes the result of `vformat(os.getloc(), fmt, args)` to `os`.

Throws: As specified in [format.err.report] or `system_error` if a call by the implementation to an operating system or other underlying API results in an error that prevents the function from meeting its specifications.

Add to Normative references [intro.refs]:

– The Unicode® Standard Version 14.0 – Core Specification

§ Appendix A: Unicode tests

This appendix gives full listings of programs for testing Unicode handling in various formatting facilities as well as test commands and their output on different platforms. The code contains additional sanity checks to ensure that the strings are encoded in some form of UTF as opposed to a legacy encoding.

C ([test.c](#)):

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    const char* message = "Привет, κόσμος!\n";
    if ((unsigned char)message[0] != 0xD0 && (unsigned char)message[1] != 0x9F)
        abort();
    printf(message);
}
```

Go ([test.go](#)):

```
package main

import "fmt"
import "log"

func main() {
    var message = "Привет, κόσμος!"
    if message[0] != 0xD0 && message[1] != 0x9F {
        log.Fatal("wrong encoding")
    }
    fmt.Println(message)
}
```

Java ([Test.java](#)):

```
class Test {
    public static void main(String[] args) {
        String message = "Привет, κόσμος!\n";
        if (message.charAt(0) != 0x41F) throw new RuntimeException();
        System.out.print(message);
    }
}
```

JavaScript / Node.js ([test.js](#)):

```
message = "Привет, κόσμος!";
if (message.charCodeAt(0) != 0x41F) throw "wrong encoding";
console.log(message);
```

Python (`test.py`):

```
message = "Привет, κόσμος!"
if ord(message[0]) != 0x41F:
    raise Exception()
print(message)
```

Rust (`test.rs`):

```
fn main() {
    if "Привет, κόσμος!".chars().nth(0).unwrap() as u32 != 0x41F {
        panic!();
    }
    println!("Привет, κόσμος!");
}
```

Linux:

```
$ cc test.c -o c-test
$ ./c-test
Привет, κόσμος!
$ ./c-test > out-c-linux.txt

$ go build -o go-test test.go
$ ./go-test
Привет, κόσμος!
$ ./go-test > out-go-linux.txt

$ java Test
Привет, κόσμος!
$ java Test > out-java-linux.txt

$ node test.js
Привет, κόσμος!
$ node test.js > out-js-linux.txt

$ python3 test.py
Привет, κόσμος!
$ python3 test.py > out-py-linux.txt

$ rustc test.rs -o rust-test
$ ./rust-test
Привет, κόσμος!
$ ./rust-test > out-rust-linux.txt
```

All output files are in UTF-8:

- [out-c-linux.txt](#)
- [out-go-linux.txt](#)
- [out-java-linux.txt](#)
- [out-js-linux.txt](#)
- [out-py-linux.txt](#)
- [out-rust-linux.txt](#)

Linux configuration:

- Ubuntu Focal 20.04 with the ru_RU.UTF-8 locale
- cc: gcc 9.3.0
- go: go1.13.8
- java: openjdk 11.0.9.1
- node: v14.5.0
- python3: 3.7.5
- rustc: 1.47.0

macOS:

```
% cc test.c -o c-test
% ./c-test
Привет, κόσμος!
% ./c-test > out-c-macos.txt

% go build -o test-go test.go
% ./test-go
Привет, κόσμος!
% ./test-go > out-go-macos.txt

% java Test
Привет, κόσμος!
% java Test > out-java-macos.txt

% node test.js
Привет, κόσμος!
% node test.js > out-js-macos.txt

% python3 test.py
Привет, κόσμος!
% python3 test.py > out-py-macos.txt
```

```
% rustc test.rs -o rust-test
% ./rust-test
Привет, κόσμος!
% ./rust-test > out-rust-macos.txt
```

All output files are in UTF-8:

- [out-c-macos.txt](#)
- [out-go-macos.txt](#)
- [out-java-macos.txt](#)
- [out-js-macos.txt](#)
- [out-py-macos.txt](#)
- [out-rust-macos.txt](#)

macOS configuration:

- macOS Catalina 10.15.7 with the ru_RU.UTF-8 locale which is the default for Russian
- cc: Apple clang version 12.0.0 (clang-1200.0.32.27)
- go: go1.15.5
- java: openjdk 14.0.1
- node: v14.5.0
- python3: 3.7.5
- rustc: 1.47.0

Windows:

```
>cl /Fe:c-test.exe test.c
...
>c-test
Привет, κόσμος!
>c-test > out-c-windows.txt
>c-test | findstr ,
Привет, κόσμος!

>go build -o go-test.exe test.go
>go-test
Привет, κόσμος!
>go-test > out-go-windows.txt
>go-test | findstr ,
Привет, κόσμος!
```

```
>java Test
Привет, ??????!
```

`>java Test > out-java-windows.txt`

`>java Test | findstr ,`

⌚ЁштхЄ, ??????!


```
>node test.js
Привет, κόσμος!
```

`>node test.js > out-js-windows.txt`

`>node test.js | findstr ,`

⌚ЯтА⌚⌚⌚⌚⌚тВ, ⌚⌚⌚⌚⌚М⌚Г⌚⌚⌚⌚⌚Б!


```
>python test.py
Привет, κόσμος!
```

`>python test.py > out-py-windows.txt`

Traceback (most recent call last):

File "...\\test.py", line 4, in <module>

print(message)

File "...\\Python39\\lib\\encodings\\cp1251.py", line 19, in encode

return codecs.charmap_encode(input,self.errors,encoding_table)[0]

UnicodeEncodeError: 'charmap' codec can't encode characters in position 8-13: chara

`>python test.py | findstr ,`

Traceback (most recent call last):

File "...\\test.py", line 4, in <module>

print(message)

File "...\\Python39\\lib\\encodings\\cp1251.py", line 19, in encode

return codecs.charmap_encode(input,self.errors,encoding_table)[0]

UnicodeEncodeError: 'charmap' codec can't encode characters in position 8-13: chara


```
>rustc test.rs -o rust-test.exe
>rust-test
Привет, κόσμος!
```

`>rust-test > out-rust-windows.txt`

`>rust-test | findstr ,`

⌚ЯтА⌚⌚⌚⌚⌚тВ, ⌚⌚⌚⌚⌚М⌚Г⌚⌚⌚⌚⌚Б!

C, JavaScript (node.js), Rust and Go produced valid UTF-8 when the output was redirected to files. Java produced a file in the legacy CP1251 encoding with `?` for non-representable code points. Python failed on transcoding to CP1251. Output files:

- [out-c-windows.txt](#)
- [out-go-windows.txt](#)
- [out-java-windows.txt](#)
- [out-js-windows.txt](#)

- [out-py-windows.txt](#)
- [out-rust-windows.txt](#)

Windows configuration:

- Windows 10 10.0.19041 with Russian Region and Language settings
- cl: Microsoft (R) C/C++ Optimizing Compiler Version 19.28.29335
- go: go1.15.6
- java: Java HotSpot(TM) 64-Bit Server VM (build 15.0.1+9-18)
- node: v14.15.3
- python: 3.9.1
- rustc: v14.15.3

§ 17. Acknowledgements

Thanks to Corentin Jabot for his work on text encodings in C++ and in particular [\[P1885\]](#) that will simplify implementation of the current proposal.

Thanks to Roger Orr, Peter Brett, Hubert Tong, the BSI C++ panel and Tom Honermann for their feedback, support, constructive criticism and contributions to the proposal.

§ References

§ Informative References

[CODESEARCH]

Andrew Tomazos. [Code search engine website](#). URL: <https://codesearch.isocpp.org>

[DOTNET-WRITE]

[.NET documentation, Console.Write Method](#). URL: <https://docs.microsoft.com/en-us/dotnet/api/system.console.write>

[ENCODING-TRENDS]

[Historical yearly trends in the usage statistics of character encodings for websites](#). URL: https://w3techs.com/technologies/history_overview/character_encoding/ms/y

[FMT]

Victor Zverovich; et al. [The fmt library](#). URL: <https://github.com/fmtlib/fmt>

[FOLLY]

[Folly: Facebook Open-source Library](#). URL: <https://github.com/facebook/folly>

[FORMAT]

Thomas Köppe. [Working Draft, Standard for Programming Language C++, Formatting \[format\]](https://wg21.link/n4901#section.20.20). URL: <https://wg21.link/n4901#section.20.20>

[GO-FMT]

Go Package Documentation, Package `fmt`. URL: <https://golang.org/pkg/fmt/>

[GOOGLE-BENCH]

Google Benchmark: A microbenchmark support library. URL: <https://github.com/google/benchmark>

[ISO2022]

ISO/IEC 2022 Information technology—Character code structure and extension techniques. URL: <https://www.iso.org/standard/22747.html>

[JAVA-PRINT]

Java™ Platform, Standard Edition 7 API Specification, Class `PrintStream`. URL: <https://docs.oracle.com/javase/7/docs/api/java/io/PrintStream.html>

[LLVM-OSTREAM]

The LLVM Compiler Infrastructure repository, `raw_ostream`. URL: https://github.com/llvm-mirror/llvm/blob/master/lib/Support/raw_ostream.cpp

[MSVC-UTF8]

Visual C++ Documentation, `/utf-8` (Set Source and Executable character sets to UTF-8). URL: <https://docs.microsoft.com/en-us/cpp/build/reference/utf-8-set-source-and-executable-character-sets-to-utf-8>

[N0147]

ISO/IEC IS 1989:2001 – Programming language COBOL, 14.8.10 `DISPLAY` statement. URL: https://web.archive.org/web/20020124065139/http://www.ncits.org/tc_home/j4htm/cobolv200112.zip

[N2162]

ISO/IEC 1539-1:2018 Information technology — Programming languages — Fortran.

[N2176]

ISO/IEC 9899:2017 Programming languages — C, 7.21.6.3. The `fprintf` function.

[P0645]

Victor Zverovich. [Text Formatting](https://wg21.link/p0645). URL: <https://wg21.link/p0645>

[P1868]

Victor Zverovich; Zach Laine. 🦄 [width: clarifying units of width and precision in `std::format`](https://wg21.link/p1868). URL: <https://wg21.link/p1868>

[P1885]

Corentin Jabot. [Naming Text Encodings to Demystify Them](https://wg21.link/p1885). URL: <https://wg21.link/p1885>

[P2216]

Victor Zverovich. [std::format improvements](https://wg21.link/p2216). URL: <https://wg21.link/p2216>

[P2418]

Victor Zverovich. [Add support for `std::generator-like` types to `std::format`](https://wg21.link/p2418). URL: <https://wg21.link/p2418>

[PERL-PRINTF]

Perl 5 version 30.0 documentation, Language reference, `printf`. URL: <https://perldoc.perl.org/functions/printf.html>

[PHP-PRINTF]

[PHP Manual, Function Reference, printf](https://www.php.net/manual/en/function.printf.php). URL: <https://www.php.net/manual/en/function.printf.php>

[PY-FUNC]

[The Python Standard Library, Built-in Functions](https://docs.python.org/3/library/functions.html). URL: <https://docs.python.org/3/library/functions.html>

[R-PRINT]

The R Core Team. [R: A Language and Environment for Statistical Computing, Reference Index, printf](https://cran.r-project.org/doc/manuals/r-release/fullrefman.pdf#page=457). URL: <https://cran.r-project.org/doc/manuals/r-release/fullrefman.pdf#page=457>

[RUBY-PRINT]

[Documentation for Ruby, print](https://docs.ruby-lang.org/en/2.7.0/ARGF.html#method-i-print). URL: <https://docs.ruby-lang.org/en/2.7.0/ARGF.html#method-i-print>

[RUST-PRINT]

[The Rust Standard Library, Macro std::print](https://doc.rust-lang.org/std/macro.print.html). URL: <https://doc.rust-lang.org/std/macro.print.html>

[RUST-STDIO]

[The Rust Programming Language repository, windows_stdio](https://github.com/rust-lang/rust/blob/db492ecd5ba6bd82205612cebb9034710653f0c2/library/std/src/sys/windows/stdio.rs). URL: <https://github.com/rust-lang/rust/blob/db492ecd5ba6bd82205612cebb9034710653f0c2/library/std/src/sys/windows/stdio.rs>

[SWIFT-PRINT]

[Swift Standard Library, print](https://developer.apple.com/documentation/swift/1541053-print). URL: <https://developer.apple.com/documentation/swift/1541053-print>

[WHATWG-CONSOLE]

[WHATWG Standards, Console](https://console.spec.whatwg.org/). URL: <https://console.spec.whatwg.org/>